



CAPABILITY-CENTRIC ARCHITECTURE

Quick Reference Cards

A comprehensive guide to implementing CCA - the unified architectural pattern for embedded and enterprise systems. These cards provide quick access to core concepts, best practices, and implementation guidelines.

11 Professional Reference Cards
Complete Implementation Guide

Version 2.0 • December 2025
Michael Stal



Core Concepts & Capability Nucleus

Understanding the Foundation of CCA

Card 1 of 11



What is a Capability?

Definition: A cohesive set of functionality that delivers tangible value.

GOOD Examples:

- ✓ Product Catalog manages product information
- ✓ Payment Processing handles transactions
- ✓ Motor Control regulates speed/position
- ✓ User Authentication manages identity

AVOID:

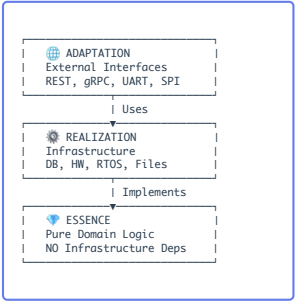
- ✗ Database-Access-Capability
- ✗ UI-Capability
- ✗ Team-A-Capability



GOLDEN RULE: Express purpose in **ONE sentence**. Focus on **WHAT**, not **HOW**. Based on **functionality**, not layers.



The Nucleus (3 Layers)



Layer Details

Layer	Purpose	Dependencies
ESSENCE	Define WHAT	Only Contracts
REALIZATION	Implement HOW	Essence + Infra
ADAPTATION	External API	Realization



Essence Layer

Contains:

- Pure domain logic
- Core algorithms
- Business rules
- Domain state

Rules:

- NO database calls
- NO file I/O
- NO hardware access
- ONLY other Capability Contracts



Realization Layer

Contains: Database access, Hardware drivers, File I/O, Message queues, RTOS calls, Cache management

Examples: PostgreSQL queries, GPIO/ADC access, Redis caching, Kafka producers



Adaptation Layer

Contains: REST controllers, gRPC services, Message queue listeners, UART/SPI handlers, WebSocket endpoints

Examples: `@RestController` classes, gRPC service implementations, MQTT subscribers, CAN bus handlers



KEY INSIGHT: Essence is pure and reusable. Realization adapts to environment. Adaptation exposes to world.



Capability Contracts

Provisions, Requirements & Protocols

Card 2 of 11

PROVISIONS

Interfaces I *PROVIDE* to others

Structure:

```
new Provision(  
  "ServiceName",  
  ServiceInterface.class,  
  "Description of what it does"  
)
```

Example Interface:

```
public interface ProductService {  
  Product getProductDetails(String id);  
  List<Product> searchProducts(String query);  
}  
  
// Declaration:  
new Provision(  
  "ProductLookup",  
  ProductService.class,  
  "Provides product details and search"  
)
```

REQUIREMENTS

Interfaces I *NEED* from others

Structure:

```
new Requirement(  
  "ServiceName",  
  ServiceInterface.class,  
  false, // optional?  
  "Why I need this"  
)
```

Example:

```
new Requirement(  
  "PricingService",  
  PricingService.class,  
  false, // REQUIRED  
  "Needed for product prices"  
)
```

PROTOCOLS

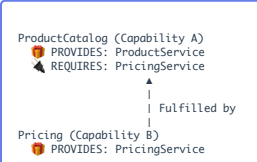
HOW to interact with me

```
new Protocol(  
  "REST_HTTP",  
  "JSON",  
  Map.of("latency", "50ms",  
         "security", "OAuth2")  
)
```

Complete Contract Example

```
CapabilityContract contract =  
  new CapabilityContract(  
  
    // PROVISIONS  
    List.of(  
      new Provision(  
        "ProductLookup",  
        ProductService.class,  
        "Product details & search"  
      ),  
      new Provision(  
        "ProductEvents",  
        ProductEventPublisher.class,  
        "Product change events"  
      )  
    ),  
  
    // REQUIREMENTS  
    List.of(  
      new Requirement(  
        "PricingService",  
        PricingService.class,  
        false, // Required  
        "For product prices"  
      ),  
      new Requirement(  
        "InventoryService",  
        InventoryService.class,  
        false, // Required  
        "For stock levels"  
      )  
    ),  
  
    // PROTOCOLS  
    List.of(  
      new Protocol("REST_HTTP",  
        Map.of("latency", "50ms",  
               "security", "OAuth2")  
      ),  
      new Protocol("gRPC", "Protobuf",  
        Map.of("latency", "20ms",  
               "security", "TLS")  
      ),  
      new Protocol("DirectCall",  
        Map.of("latency", "5ms")  
      )  
    )  
  );
```

Interaction Pattern



💡 **CONTRACT DESIGN PRINCIPLES:** ☒ Focus on **WHAT** (outcome), not **HOW** (implementation) • ☒ Use Semantic Versioning (**MAJOR.MINOR.PATCH**) • ☒ Keep stable yet flexible • ☒ Support multiple protocols

Capability Lifecycle

Initialization, Dependencies & State Management

Card 3 of 11

CapabilityInstance Interface

```
public interface CapabilityInstance {  
  
    // Phase 1: Setup  
    void initialize();  
  
    // Phase 2: Begin work  
    void start();  
  
    // Phase 3: Stop work  
    void stop();  
  
    // Phase 4: Release resources  
    void cleanup();  
  
    // Provide contract implementation  
    Object getContractImplementation(  
        Class<?> contractType  
    );  
  
    // Receive dependencies  
    void injectDependency(  
        Class<?> contractType,  
        Object implementation  
    );  
  
}
```

Lifecycle States



INSTANTIATED
+ initialize()



INITIALIZED
+ injectDependency()



DEPENDENCIES INJECTED
+ start()



STARTED (Runtime)
+ stop()



STOPPED
+ cleanup()



CLEANED UP

Manager Responsibilities

CapabilityLifecycleManager:

1. Build dependency graph
2. Topological sort (correct order)
3. Initialize capabilities
4. Inject dependencies
5. Start capabilities
6. (Runtime operations)
7. Stop capabilities (reverse order)
8. Cleanup

Implementation Example

```
public class ProductCatalogCapability  
    implements CapabilityInstance {  
  
    private final ProductCatalogEssence essence;  
    private final DatabasePool database;  
  
    // Dependencies (injected later)  
    private PricingService pricingService;  
    private InventoryService inventoryService;  
  
    @Override  
    public void initialize() {  
        // Setup internal state  
        database.initializeSchema();  
        essence.loadInitialData();  
    }  
  
    @Override  
    public void start() {  
        // Begin active operations  
        startBackgroundTasks();  
        openConnections();  
    }  
  
    @Override  
    public void stop() {  
        // Graceful shutdown  
        stopBackgroundTasks();  
        closeConnections();  
    }  
  
    @Override  
    public void cleanup() {  
        // Release resources  
        database.close();  
        cache.clear();  
    }  
  
    @Override  
    public Object getContractImplementation(  
        Class<?> contractType) {  
        if (contractType ==  
            ProductService.class) {  
            return new ProductServiceImpl(essence, database, pricingService, inventoryService);  
        }  
        return null;  
    }  
  
    @Override  
    public void injectDependency(  
        Class<?> contractType,  
        Object implementation) {  
        if (contractType ==  
            PricingService.class) {  
            this.pricingService =  
                (PricingService) implementation;  
        }  
        // ... other dependencies  
    }  
  
}
```

 **KEY INSIGHT:** Manager handles orchestration automatically. You just implement the interface. Dependencies are injected in correct order based on Contract declarations.

⚡ Efficiency Gradients

Performance Optimization Where You Need It

Card 4 of 11

🎯 Core Concept

Not all code needs same optimization!

Different parts operate at different levels of performance and abstraction.

📊 The Gradient Spectrum

HIGH EFFICIENCY → LOW EFFICIENCY
LOW ABSTRACTION → HIGH ABSTRACTION

- 
 - Bare-metal code
 - Direct HW registers
 - Assembly
 - RTOS interrupts

⌚ Latency: Microseconds

🚫 Use: Motor control, sensor interrupts

- 
 - Standard OS services
 - Compiled languages
 - Optimized libraries

⌚ Latency: Milliseconds

🚫 Use: Business logic, API calls, DB queries

- 
 - Frameworks (Spring, etc.)
 - GC languages (Java, etc.)
 - ORMs, JSON parsing

⌚ Latency: Seconds (OK)

🚫 Use: Config, logging, admin tasks

🎨 When to Use Each

Gradient	Use For
High	Real-time loops, interrupts, HFT
Medium	Most application code
Low	Config, logging, admin

🖨️ Example: Motor Control

🔥 HIGH EFFICIENCY (Critical):

```
// Runs every 100 microseconds
private void onMotorInterrupt() {
    // Direct HW register access
    int pos = hardwareRegisters
        .readEncoder();

    // Inline calculation
    double speed = stateEstimator
        .estimateSpeed(pos);

    // Optimized PID (no allocations)
    double output = pidController
        .calculate(speed, target);

    // Direct PWM write
    pwmDriver.setDutyCycle(output);
}
```

⌚ Latency: ~10 microseconds

🌈 LOW EFFICIENCY (Non-Critical):

```
// Runs on-demand (rarely)
public void updateConfiguration(
    String configData) {

    // JSON parsing (high-level)
    JSONObject config =
        new JSONParser().parse(configData);

    // Network call
    Params params = httpClient
        .get("/api/motor-params");

    // Update essence
    essence.setTargetSpeed(
        config.getDouble("targetSpeed")
    );

    // Database persistence
    database.save("motor_config", config);
}
```

⌚ Latency: ~500 milliseconds (acceptable)

💡 KEY INSIGHT: Both methods can exist in the SAME Capability! Apply the right gradient to each function.
Don't over-optimize non-critical paths. Don't under-optimize critical paths.

Evolution Envelopes

Managing Change with Predictability

Card 5 of 11

Semantic Versioning

MAJOR . MINOR . PATCH
2 . 1 . 3

Component	When to Increment	Impact
MAJOR	Breaking changes	Incompatible API
MINOR	New features	Backward compatible
PATCH	Bug fixes	Backward compatible

Examples

MAJOR (2.x.x → 3.0.0):

- Remove method from interface
- Change method signature
- Change data format
- Remove entire provision

MINOR (2.1.x → 2.2.0):

- Add new method to interface
- Add new provision
- Add optional parameter
- Enhance existing feature

PATCH (2.1.3 → 2.1.4):

- Fix bug in logic
- Performance improvement
- Security patch
- Documentation fix

Evolution Envelope

```
public class EvolutionEnvelope {  
    String currentVersion;  
    String previousVersion;  
    List<DeprecationPolicy> deprecations;  
    List<MigrationPath> migrations;  
}
```

Deprecation Policy

```
new DeprecationPolicy(  
    "Contract v1.x",           // What  
    LocalDate.of(2025, 12, 31), // EOL  
    "Performance improvements", // Why  
    "Use Contract v2.x"       // Replacement  
)
```

Components:

- Target:** What's being deprecated
- Warning period:** How long supported
- EOL date:** When support ends
- Reason:** Why deprecating
- Replacement:** What to use instead

Migration Path

```
new MigrationPath(  
    "1.5.0",           // From  
    "2.0.0",           // To  
    "https://docs.../migration", // Docs  
    "upgrade-script.sh" // Tool  
)
```

Includes:

- Code examples
- Step-by-step guide
- Automated scripts
- Testing recommendations
- Rollback procedures

Version Strategy

DO:

- ✔ Plan breaking changes carefully
- ✔ Provide migration paths
- ✔ Give adequate warning period
- ✔ Support N-1 versions
- ✔ Document all changes

DON'T:

- ✗ Break compatibility without notice
- ✗ Skip version numbers
- ✗ Change MAJOR version frequently
- ✗ Deprecate without replacement

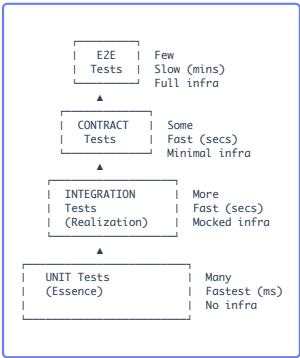
💡 **KEY INSIGHT:** Evolution Envelopes make change predictable. Consumers know what to expect. Migrations are smooth. System stability is maintained.

Testing Strategy

The Testing Pyramid for CCA

Card 6 of 11

The Pyramid



Test Layer Details

Layer	Speed	Infra	Focus
Unit	ms	None	Pure logic
Integration	secs	Mocked	Infra integration
Contract	secs	Minimal	Interface compliance
E2E	mins	Full	Complete workflows

Quantity Guide

- Unit: (100s)
- Integration: (10s)
- Contract: (5-10)
- E2E: (1-5)

Unit Test (Essence)

```
@Test
void testProductSearch() {
    // Arrange
    ProductCatalogEssence essence =
        new ProductCatalogEssence();
    essence.addProduct(
        new Product("1", "Laptop", 999.99)
    );

    // Act
    List<Product> results =
        essence.searchProducts("Laptop");

    // Assert
    assertEquals(1, results.size());
    assertEquals("Laptop",
        results.get(0).name());
}
```

Fast, no dependencies

Integration Test (Realization)

```
@Test
void testDatabaseIntegration() {
    // Arrange (with mocked DB)
    DatabasePool mockDb = mock(DatabasePool);
    ProductCatalogRealization realization =
        new ProductCatalogRealization(
            essence, mockDb
        );

    when(mockDb.query(anyString()))
        .thenReturn(testProduct);

    // Act
    Product result =
        realization.getProduct("1");

    // Assert
    assertNotNull(result);
    verify(mockDb).query(contains("SELECT"));
}
```

Tests infrastructure integration

Contract Test

```
@Test
void testContractCompliance() {
    ProductService service =
        capability.getContractImplementation(
            ProductService.class
        );

    // Verify all methods exist
    assertNotNull(service);
    assertNotNull(
        service.getProductDetails("1")
    );
    assertNotNull(
        service.searchProducts("test")
    );
}
```

Verifies interface compliance

KEY INSIGHT: Most tests should be Unit tests (Essence). They're fast, reliable, and test pure logic. Integration tests verify infrastructure. Contract tests ensure compatibility. E2E tests validate complete workflows.

Packaging Strategies

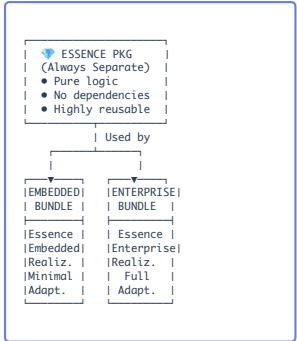
Bundles & Deployment Modes

Card 7 of 11

Golden Rules

1. **ALWAYS** package Essence separately
2. Create **feature-based bundles**
3. Use **dependency injection**

Bundle Structure



Embedded Bundle

Contains:

- Essence (pure logic)
- Embedded Realization:
 - Direct HW access
 - RTOS calls
 - GPIO/ADC/Timers
- Minimal Adaptation:
 - UART, SPI, CAN Bus, I2C

Use for: IoT devices, Microcontrollers, Industrial control, Automotive systems

Enterprise Bundle

Contains:

- Essence (same pure logic)
- Enterprise Realization:
 - PostgreSQL/MySQL
 - Redis cache
 - Kafka/RabbitMQ
 - Elasticsearch
- Full Adaptation:
 - REST API, gRPC, GraphQL, WebSockets

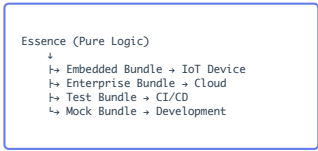
Use for: Cloud platforms, Web applications, Microservices, SaaS products

Deployment Modes

Mode	Use Case
Monolithic	Development, debugging
Microservice	Production, scalability
Containerized	Cloud-native, K8s
Serverless	Cost optimization
Embedded	IoT, hardware

Key Benefit

Same Essence code runs everywhere!



Maven/Gradle Structure

```
product-catalog/
├── essence/           (JAR)
├── embedded/         (JAR)
├── enterprise/       (JAR)
└── tests/            (JAR)
```

KEY INSIGHT: Essence is environment-agnostic. Realization adapts to infrastructure. Adaptation exposes appropriate interfaces. One codebase, multiple deployments.

Embedded vs Enterprise

Comparing Implementation Approaches

Card 8 of 11

Embedded Systems

Realization Layer:

- Direct hardware registers
- RTOS (FreeRTOS, Zephyr)
- GPIO, ADC, DAC, PWM
- Hardware timers
- Interrupt handlers
- DMA controllers
- Watchdog timers

Adaptation Layer:

- UART (serial)
- SPI (sensors)
- I2C (peripherals)
- CAN Bus (automotive)
- Modbus (industrial)
- BLE (wireless)

Efficiency Gradients:

- 🔥 High efficiency critical
- Bare-metal for real-time
- Microsecond latency
- Direct HW access

Resource Management:

- Explicit memory limits
- CPU allocation (%)
- Power budgets
- Stack/heap sizes

Testing:

- Hardware-in-the-loop (HIL)
- Simulator tests
- Oscilloscope verification
- Timing analysis

Deployment:

- Single binary
- Firmware update
- OTA (Over-The-Air)
- Flash memory

Example Capabilities:

- Motor Control
- Sensor Reading
- Temperature Regulation
- Power Management

Enterprise Systems

Realization Layer:

- PostgreSQL, MySQL
- MongoDB, Cassandra
- Redis, Memcached
- Kafka, RabbitMQ
- Elasticsearch
- S3, Blob Storage
- HTTP clients

Adaptation Layer:

- REST APIs
- gRPC services
- GraphQL endpoints
- WebSockets
- Server-Sent Events
- SOAP (legacy)

Efficiency Gradients:

- ⚙️ Medium efficiency OK
- Frameworks acceptable
- Millisecond latency
- Higher abstractions

Resource Management:

- Auto-scaling policies
- Horizontal scaling
- Load balancing
- Container limits (K8s)

Testing:

- Integration tests
- API contract tests
- Load testing
- Chaos engineering

Deployment:

- Microservices
- Docker containers
- Kubernetes pods
- Serverless functions
- CI/CD pipelines

Example Capabilities:

- Product Catalog
- Payment Processing
- User Authentication
- Order Management

Common Ground (Same for Both)

Aspect	Embedded & Enterprise
Essence	Pure domain logic, NO infrastructure dependencies
Contracts	Provisions, Requirements, Protocols
Lifecycle	initialize() → start() → stop() → cleanup()
Evolution	Semantic versioning, deprecation policies
Testing	Unit tests for Essence (fast, no infrastructure)

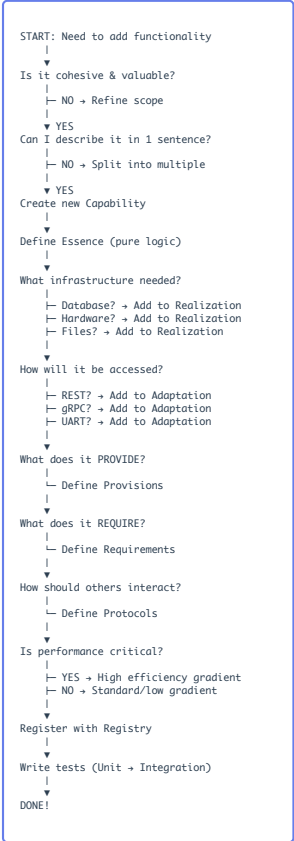
💡 **KEY INSIGHT:** CCA provides ONE architectural pattern for BOTH worlds. Essence stays the same. Realization and Adaptation change based on environment. This enables true code reuse across embedded and enterprise.

Decision Tree & Checklist

Step-by-Step Implementation Guide

Card 9 of 11

Decision Tree



New Capability Checklist

- Design Phase:**
- Capability name describes cohesive functionality
 - Purpose expressible in ONE sentence
 - Not based on technical layer
 - Not based on organizational boundary
 - Clear value proposition

- Essence Phase:**
- Pure domain logic only
 - NO infrastructure dependencies
 - NO database calls
 - NO file I/O
 - NO hardware access
 - Only depends on other Contracts

- Realization Phase:**
- Infrastructure integration identified
 - Database access (if needed)
 - Hardware drivers (if needed)
 - File I/O (if needed)
 - Message queues (if needed)

- Adaptation Phase:**
- External interfaces defined
 - REST API (if needed)
 - gRPC (if needed)
 - UART/SPI (if needed)
 - Message queue listeners (if needed)

- Contract Phase:**
- Provisions defined (what I provide)
 - Requirements defined (what I need)
 - Protocols defined (how to interact)
 - Optional vs required marked
 - Descriptions written

- Lifecycle Phase:**
- CapabilityInstance implemented
 - initialize() implemented
 - start() implemented
 - stop() implemented
 - cleanup() implemented
 - getContractImplementation() implemented
 - injectDependency() implemented

- Testing Phase:**
- Unit tests for Essence (many)
 - Integration tests for Realization (some)
 - Contract tests (verify interfaces)
 - E2E tests (few, critical paths)

- Packaging Phase:**
- Essence packaged separately
 - Feature bundles created
 - Dependencies declared

- Documentation Phase:**
- EvolutionEnvelope created
 - Version number assigned
 - Migration paths documented
 - API documentation written

- Deployment Phase:**
- Registered with CapabilityRegistry
 - Deployment mode selected
 - Resource requirements declared (embedded)
 - Monitoring configured

PRO TIP: Use this checklist for every new Capability. It ensures you don't miss critical steps and maintain architectural consistency.

✓ Patterns & ✗ Anti-Patterns

Best Practices & Common Mistakes

Card 10 of 11

✓ DO - Best Practices

Design:

- ✓ Define by functional cohesion
- ✓ One clear responsibility
- ✓ Express in one sentence
- ✓ Deliver tangible value

Essence:

- ✓ Keep pure (no infrastructure)
- ✓ High test coverage
- ✓ Reusable across environments
- ✓ Only depend on Contracts

Contracts:

- ✓ Use Semantic Versioning
- ✓ Document breaking changes
- ✓ Provide migration paths
- ✓ Support multiple protocols

Efficiency:

- ✓ Apply gradients strategically
- ✓ Optimize critical paths
- ✓ Use abstractions elsewhere
- ✓ Measure before optimizing

Testing:

- ✓ Many unit tests (Essence)
- ✓ Some integration tests
- ✓ Contract compliance tests
- ✓ Few E2E tests

Packaging:

- ✓ Separate Essence package
- ✓ Feature-based bundles
- ✓ Dependency injection
- ✓ Clear module boundaries

Evolution:

- ✓ Plan breaking changes
- ✓ Deprecate gracefully
-
- ✓ Support N-1 versions
- ✓ Document migrations

✗ DON'T - Anti-Patterns

Design:

- ✗ Create by technical layer
- ✗ Create by team structure
- ✗ Too many responsibilities
- ✗ Unclear value proposition

Essence:

- ✗ Mix infrastructure code
- ✗ Direct database calls
- ✗ File I/O in Essence
- ✗ Hardware access in Essence

Contracts:

- ✗ Break without notice
- ✗ Skip version numbers
- ✗ Change frequently (MAJOR)
- ✗ Undocumented changes

Efficiency:

- ✗ Over-optimize everything
- ✗ Under-optimize critical paths
- ✗ Ignore performance data
- ✗ Premature optimization

Testing:

- ✗ Only E2E tests
- ✗ No unit tests
- ✗ Skip contract tests
- ✗ Test with real infrastructure

Packaging:

- ✗ Mix Essence with Realization
- ✗ Monolithic packages
- ✗ Tight coupling
- ✗ Circular dependencies

Evolution:

- ✗ Breaking changes without warning
- ✗ No deprecation period
-
- ✗ Missing migration docs
- ✗ Support too many versions

📖 Common Mistakes

Mistake	Fix
Circular Dependencies A requires B, B requires A	Extract shared logic to C
God Capability One Capability does everything	Split into cohesive units
Anemic Essence Essence has no logic	Move business rules to Essence
Infrastructure in Essence Database calls in Essence	Move to Realization
Missing Contracts Direct coupling	Define explicit Contracts

⚠ **WARNING:** Anti-patterns lead to tight coupling, difficult testing, and brittle architecture. Follow the DO list, avoid the DON'T list, and your system will be maintainable and evolvable.

One-Page Summary

The Essentials at a Glance

Card 11 of 11

The 3-Layer Nucleus

- ADAPTATION → External interfaces (REST, gRPC, UART)
 - ↳ uses
- REALIZATION → Infrastructure (DB, HW, Files, MQ)
 - ↳ implements
- ESSENCE → Pure domain logic (NO infrastructure)

The Contract (3 Parts)

```
CapabilityContract {  
  🏠 Provisions    // What I PROVIDE  
  📋 Requirements // What I NEED  
  🛠️ Protocols    // HOW to interact  
}
```

The Lifecycle (6 Phases)

- 🏠 Instantiated → 🏠 Initialized
- + 📋 Dependencies Injected + 🛠️ Started
- + 🛑 Stopped + 🧹 Cleaned Up

Efficiency Gradients

- 🔥 Critical Path → High efficiency, low abstraction
- ⚙️ Standard Path → Balanced
- 🌈 Non-Critical → High abstraction, low efficiency

Semantic Versioning

MAJOR.MINOR.PATCH
2 . 1 . 3

MAJOR: Breaking changes
MINOR: New features (compatible)
PATCH: Bug fixes (compatible)

Testing Pyramid

E2E (Few)
Contract (Some)
Integration (More)
Unit/Essence (Many)

Packaging

Essence (Separate) → Embedded Bundle
→ Enterprise Bundle
→ Test Bundle

Key Principles

- Capabilities** = Cohesive functionality
- Essence** = Pure logic, no infrastructure
- Contracts** = Explicit dependencies
- Gradients** = Optimize where needed
- Evolution** = Predictable change
- Testing** = Pyramid (many unit, few E2E)
- Packaging** = Essence separate
- Deployment** = Same code, any environment

Decision Flowchart

New Feature? → Cohesive?
→ Define Capability
→ Pure Logic (Essence)
→ Infrastructure (Realization)
→ External API (Adaptation)
→ Define Contract (P/R/P)
→ Implement Lifecycle
→ Write Tests
→ Register & Deploy

Remember

✅ DO	❌ DON'T	🎯 GOAL
Separate concerns Explicit contracts Test Essence	Mix layers Circular deps Skip tests	Maintainable Evolvable Performant systems

YOU'RE READY TO BUILD WITH CCA!

Keep these cards handy for quick reference

Version 2.0 • December 2025 • Michael Stal